

Molfig

Static molecular structure rendering for Typst

v0.1.4

2026-09-07

MIT

Render molecular structures through [maquette](#).

Eito Yoneyama

Molfig turns PDB, mmCIF, BinaryCIF, and XYZ structures into static OBJ, STL, or PLY meshes inside a Rust WebAssembly plugin, then hands those meshes to [maquette](#) for document rendering.

Table of Contents

Overview	2	V.1 Choosing A Mesh Format	36
I.1 Design Goals	2	Metadata Reference	37
I.2 Rendering Pipeline	2	VI.1 Assembly And Unit Metadata . . .	39
Quickstart	4	Practical Workflows	40
II.1 Reading Structure Files	5	VII.1 A Publication Figure	40
II.2 A Complete XYZ Example	6	VII.2 A Lightweight Draft Figure	40
II.3 XYZ Validation Corpus	9	VII.3 Export For External Tools	41
II.4 Choosing The First Options	10	Troubleshooting	42
Public API	11	License And Notices	43
III.1 Rendering Commands	11	Development	45
III.2 Export Commands	14	Index	46
III.3 Metadata Commands	16		
Shared Mesh Options	17		
IV.1 Input Selection	17		
IV.2 Structure Selection	17		
IV.3 Representation Selection	18		
IV.3.1 Cartoon, Spacefill, And			
Surface	19		
IV.3.2 Viewer Theme Overrides	20		
IV.3.3 Annotation Color Rules	29		
IV.3.4 Chain ID Color Rules	30		
IV.4 Geometry And Quality	32		
IV.4.1 Semantic Decimate	33		
Maquette Rendering	36		

Part I

Overview

Molfig is a Typst package for molecular figures in static documents. It accepts PDB, mmCIF, BinaryCIF, and XYZ input, converts the structure into a CPU-side Mol*-style Model/Structure/Unit representation, builds static geometry, exports the mesh as OBJ, STL, or PLY, and delegates final rendering to maquette.

Mol* is an open-source molecular visualization toolkit and web viewer. Molfig uses Mol* as the compatibility target for structure interpretation, static geometry, and OBJ/STL export behavior; see molstar.org¹ and [molstar/molstar](https://github.com/molstar/molstar)².

I.1 Design Goals

Molfig is designed around four constraints:

1. Typst packages cannot invent paths into the caller's project. On Typst 0.15.0 or later, the caller may pass a project-side `path(...)` value and Molfig will read it as bytes. For Typst 0.14-compatible documents, the caller should pass bytes produced by `read(..., encoding: none)`.
2. The plugin should run in Typst's WebAssembly environment without WebGL.
3. The molecular path should preserve Mol*-style structure concepts rather than flattening everything at parse time.
4. The final image should be a document-native static rendering through maquette, not an interactive viewer snapshot.

Molfig generates static meshes and is not a Mol* WebGL renderer. `representation: "surface"` computes the Mol* molecular-surface field and marching-cubes mesh on the CPU. Gaussian volume and density-map visuals remain outside the static export contract; ViewerAuto's Huge and Gigantic Gaussian surfaces are likewise generated on the CPU.

I.2 Rendering Pipeline

Stage	Control	Role
Read	<code>path(...)</code> or <code>read(..., encoding: none)</code>	The user document supplies either a Typst 0.15+ project path or already-read structure bytes.
Parse	<code>format</code>	The plugin parses PDB, text CIF/mmCIF, BinaryCIF MessagePack, or XYZ coordinate data.
Model	<code>assembly</code> , <code>alt-loc</code>	Model, Structure, Unit, unit operators, altLoc policy, bonds, secondary structure, and coarse data are selected.

¹<https://molstar.org/>

²<https://github.com/molstar/molstar>

Geometry	representation, quality	The static geometry layer builds spheres, cylinders, tubes, sheets, ribbons, nucleotide visuals, carbohydrate visuals, and related mesh spans.
Export	mesh-format	The mesh is serialized as OBJ, binary STL, or ASCII PLY bytes.
Render	config	The exported bytes and maquette configuration are passed to maquette.

Part II

Quickstart

The following example renders PDB entry 9R1O. Put 9R1O.typ and 9R1O.pdb in the same directory, then compile the Typst file.

Complete 9R1O example

 9R1O.typ

```
1  #import "@preview/molfig:0.1.4"
2
3  #set page(width: auto, height: auto, margin: 0mm)
4
5  // Uses structural data from RCSB PDB / wwPDB.
6  // PDB ID: 9R1O
7  // PDB DOI: https://doi.org/10.2210/pdb9R1O/pdb
8  // PDB archive data files are available under CC0 1.0.
9  #let pdb = path("9R1O.pdb")
10
11 #molfig.render(
12   pdb,
13   format: "pdb",
14   representation: "cartoon",
15   assembly: "1",
16   mesh-format: "obj",
17   quality: "high",
18   center: true,
19   output-format: "svg",
20   config: (
21     azimuth: 35,
22     elevation: 24,
23     background: "",
24   ),
25 )
```

The `path("9R1O.pdb")` input in this example requires Typst 0.15.0 or later. On an older Typst version, replace it with `read("9R1O.pdb", encoding: none)` and pass the resulting bytes to `molfig.render`.



Figure 1: RCSB PDB entry 9R1O rendered by Molfig with the complete Quickstart settings.

Structural data source: RCSB PDB / wwPDB, PDB ID 9R1O, <https://doi.org/10.2210/pdb9R1O/pdb>³. PDB archive data files are available under CC0 1.0. Deposition authors: Petrenas, Ozga, Chubb, and Woolfson.

II.1 Reading Structure Files

On Typst 0.15.0 or later, prefer passing a `path(...)` value for file-backed structure data. The path is resolved by Typst relative to the caller's file, and Molfig reads it internally with `encoding: none`. This keeps package calls concise while preserving the caller-side project boundary.

Format	Typst 0.15+ path	Example
--------	------------------	---------

³<https://doi.org/10.2210/pdb9R1O/pdb>

PDB	<code>path("9R10.pdb")</code>	RCSB PDB entry 9R1O.
mmCIF	<code>path("1FYY.cif")</code>	RCSB PDB entry 1FYY.
BinaryCIF	<code>path("1CRN.bcif")</code>	RCSB PDB entry 1CRN.
XYZ	<code>path("molecule.xyz")</code>	A standard atom-count/comment/coordinates XYZ file.

For documents that must also compile on Typst 0.14, read the file in the caller document and pass the resulting bytes. Always use `encoding: none` for external structure files; it preserves BinaryCIF bytes and avoids Unicode decoding loss for fixed-width PDB columns.

For the same files, use `read("9R10.pdb", encoding: none)`, `read("1FYY.cif", encoding: none)`, or `read("1CRN.bcif", encoding: none)`. Complete, rendered examples for all three archive formats appear below.

These files are RCSB PDB / wwPDB entries 9R1O, 1FYY, and 1CRN. Their PDB DOIs are 9R1O⁴, 1FYY⁵, and 1CRN⁶. PDB archive data files are available under CC0 1.0.

Passing a Typst string is accepted for small inline examples. A string is treated as inline molecular text, not as a file path.

II.2 A Complete XYZ Example

The following real-data example renders ethanol from the first PubChem3D conformer for PubChem CID 702. Save these two files with the shown `data/ethanol.xyz` relative path, then compile `ethanol-xyz.typ`.

PubChem ethanol conformer in XYZ format

 `data/ethanol.xyz`

```

1  9
2  PubChem CID 702 (ethanol), PubChem3D conformer 000002BE00000001
3  O  -1.1712   0.2997   0.0000
4  C  -0.0463  -0.5665   0.0000
5  C   1.2175   0.2668   0.0000
6  H  -0.0958  -1.2120   0.8819
7  H  -0.0952  -1.1938  -0.8946
8  H   2.1050  -0.3720  -0.0177
9  H   1.2426   0.9307  -0.8704
10 H   1.2616   0.9052   0.8886
11 H  -1.1291   0.8364   0.8099

```

⁴<https://doi.org/10.2210/pdb9R1O/pdb>

⁵<https://doi.org/10.2210/pdb1FYY/pdb>

⁶<https://doi.org/10.2210/pdb1CRN/pdb>

Complete XYZ rendering example

 ethanol-xyz.typ

```
1  #import "@preview/molfig:0.1.4"
2
3  #set page(width: 82mm, height: 82mm, margin: 4mm)
4
5  // PubChem CID 702 (ethanol), PubChem3D conformer 000002BE00000001.
6  // Record: https://pubchem.ncbi.nlm.nih.gov/compound/702
7  // Retrieved through PubChem PUG REST on 2026-08-24.
8  #let xyz = path("data/ethanol.xyz")
9
10 #molfig.render(
11   xyz,
12   format: "xyz",
13   representation: "default",
14   color-theme: "element-symbol",
15   mesh-format: "obj",
16   quality: "high",
17   center: true,
18   output-format: "svg",
19   config: (
20     azimuth: 35,
21     elevation: 24,
22     background: "",
23   ),
24   width: 74mm,
25   height: 74mm,
26 )
```

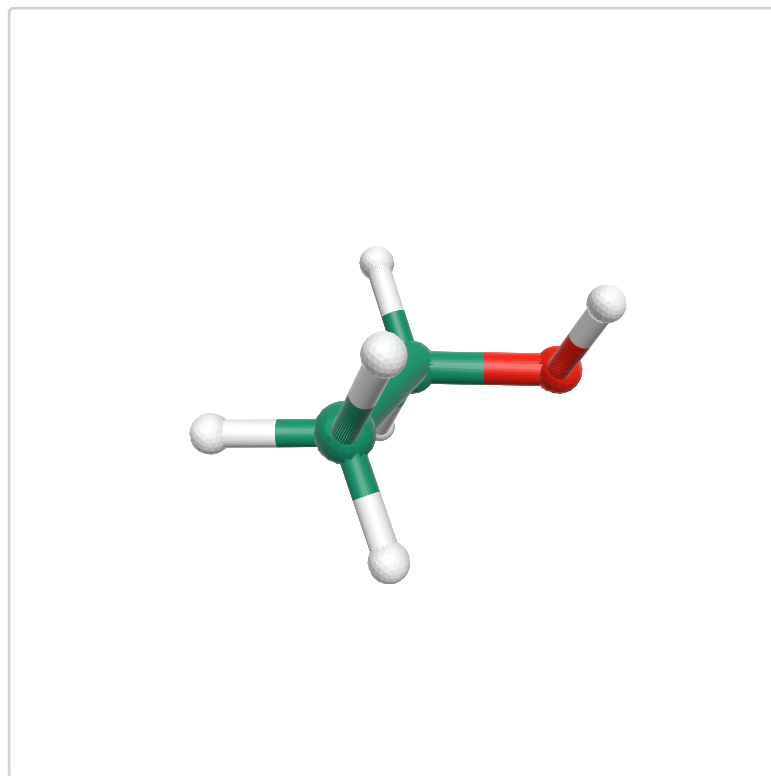


Figure 2: PubChem CID 702 (ethanol) rendered from XYZ with the complete settings above.

Coordinate source: PubChem CID 702⁷, PubChem3D conformer 000002BE00000001. The 3D SDF was retrieved through PubChem PUG REST⁸ on 2026-08-24 and transcribed to XYZ without changing atom order or coordinates. General PubChem citation: Kim et al. (2025), doi:10.1093/nar/gkae1059⁹.

⁷<https://pubchem.ncbi.nlm.nih.gov/compound/702>

⁸<https://pubchem.ncbi.nlm.nih.gov/docs/pug-rest>

⁹<https://doi.org/10.1093/nar/gkae1059>

II.3 XYZ Validation Corpus

The bundled real-data corpus expands the example to four PubChem3D conformers. All files preserve the source SDF atom order and four-decimal coordinates.

Compound	CID	Formula / atoms	Source SDF bonds
Ethanol (ethanol.xyz)	702 ¹⁰	C2H6O / 9	8
Benzene (benzene.xyz)	241 ¹¹	C6H6 / 12	12
Aspirin (aspirin.xyz)	2244 ¹²	C9H8O4 / 21	21
Caffeine (caffeine.xyz)	2519 ¹³	C8H10N4O2 / 24	25

examples/data/XYZ_VALIDATION.json pins each source URL, conformer ID, formula, source bond endpoints and orders, element composition, coordinate bounds, and SHA-256 digest. The offline validator rejects non-canonical line counts, symbols, coordinate precision, non-finite or coincident atoms, formula or bound drift, duplicate bonds, and any difference between Mol*-style XYZ bond inference and the source SDF connectivity.

The Rust regression test also compares explicit format: "xyz" parsing with auto-detection, including atom identities, coordinates, inferred endpoints, bond metadata, ring counts, and aromatic-bond counts. Finally, a real Chrome/WebGL run at pinned Mol* commit 1b8117d3f10f7c978aabb5a0d3d47370635aefe4 exports the default ball-and-stick representation. Molfig's OBJ is required to match that text exactly and its binary STL is required to match byte for byte for all four corpus records.

¹⁰<https://pubchem.ncbi.nlm.nih.gov/compound/702>

¹¹<https://pubchem.ncbi.nlm.nih.gov/compound/241>

¹²<https://pubchem.ncbi.nlm.nih.gov/compound/2244>

¹³<https://pubchem.ncbi.nlm.nih.gov/compound/2519>

II.4 Choosing The First Options

Question	Recommended setting	Why
I want the Mol* Viewer Cartoon style.	representation: "cartoon"	Uses the Viewer Quick Styles Cartoon preset: polymer cartoon plus atomic detail for ligands and other non-polymer components.
I want only the polymer cartoon.	representation: "polymer-cartoon"	Uses the Mol* Cartoon provider defaults: polymer trace, nucleotide ring, and polymer gap visuals.
I need readable geometry diffs.	mesh-format: "obj"	OBJ is text and preserves face groups.
I need compact triangle bytes.	mesh-format: "stl"	STL is binary and simple for downstream mesh tools.
I want text mesh plus group metadata.	mesh-format: "ply"	PLY is ASCII and carries package-owned group comments/properties.
Compilation is slow.	quality: "auto" or decimate: 0.3	Lets Molfig pick lower tessellation for large structures or apply semantic molecular level-of-detail.

Part III

Public API

Every public command takes `{data}` as its first positional argument. It can be bytes from `read(..., encoding: none)`, inline molecular text as a string, or a Typst 0.15+ `path(...)` value. The same mesh options are shared by `render`, `render-object`, `export`, and `metadata` commands unless noted otherwise.

Command	Returns	Use when
<code>#render</code>	content	You want the figure directly in the document.
<code>#render-object</code>	dictionary	You want the rendered content plus raw mesh bytes and metadata.
<code>#to-obj</code>	bytes	You need OBJ mesh bytes for maquette or an external tool.
<code>#to-mtl</code>	bytes	You need the companion material library for OBJ output.
<code>#to-stl</code>	bytes	You need binary STL triangle data.
<code>#to-ply</code>	bytes	You need ASCII PLY output with Molfig meta-data comments/properties.
<code>#info</code>	dictionary	You want structure and render planning meta-data without rendering.
<code>#mesh-info</code>	dictionary	You want maquette's mesh metadata for the generated OBJ/STL/PLY bytes.

III.1 Rendering Commands

```
#render(  
  {data},  
  {format},  
  {mesh-format},  
  {representation},  
  {config},  
  {width},  
  {height},  
  {output-format}  
)
```

Converts molecular bytes to a static mesh and delegates rendering to maquette.

Argument
<code>{data}</code> bytes str

Molecular structure input. Pass `path("file.pdb")` on Typst 0.15+, or `read("file.pdb", encoding: none)` for Typst 0.14-compatible documents.

Argument

`(format): "auto"`

str

Input parser selection. Use an explicit format for reproducible documents.

Argument

`(mesh-format): "obj"`

str

Intermediate mesh format sent to maquette.

Argument

`(config): (:)`

dictionary

JSON-encoded and passed to maquette. For OBJ rendering, Molfig adds the active color theme as a maquette material map; explicit `config.materials` entries override generated colors with the same material id.

Argument

`(width): "auto"`

str

Width forwarded to the maquette rendering command.

Argument

`(height): "auto"`

str

Height forwarded to the maquette rendering command.

Argument

`(output-format): "png"`

str

Output format forwarded to maquette. PNG uses maquette's Z-buffer raster output and is recommended for high-poly meshes, large assemblies, and spacefill representations. SVG preserves vector geometry, but is intended for small to moderately sized meshes because each visible mesh face contributes SVG content that Typst must parse.

#render-object(`{data}`, `{mesh-format}`)

Returns a dictionary with:

Key	Value
kind	The string "render-object".
format	The selected mesh format.
mesh_format	The selected mesh format with snake_case naming.
mesh	Generated OBJ/STL/PLY bytes.

materials	Generated maquette material map for OBJ, or an empty dictionary for STL/PLY.
info	The same dictionary returned by <code>#info</code> for the same mesh options.
content	The Typst content returned by <code>#render</code> .

Render and inspect RCSB PDB entry 1FYF

 `render-object.typ`

```

1  #import "@preview/molfig:0.1.4"
2
3  // Structural data: RCSB PDB / wwPDB entry 1FYF.
4  // https://doi.org/10.2210/pdb1FYF/pdb (CC0 1.0)
5  #let object = molfig.render-object(
6    read("1FYF.cif", encoding: none),
7    format: "cif",
8    representation: "cartoon",
9    mesh-format: "obj",
10   assembly: "1",
11   config: (azimuth: 30, elevation: 18, background: ""),
12   width: 92mm,
13   height: 64mm,
14   output-format: "svg",
15 )
16
17 #object.content
18 #align(center)[
19   Atoms: #object.info.atom_count \
20   Mesh format: #object.mesh_format
21 ]

```

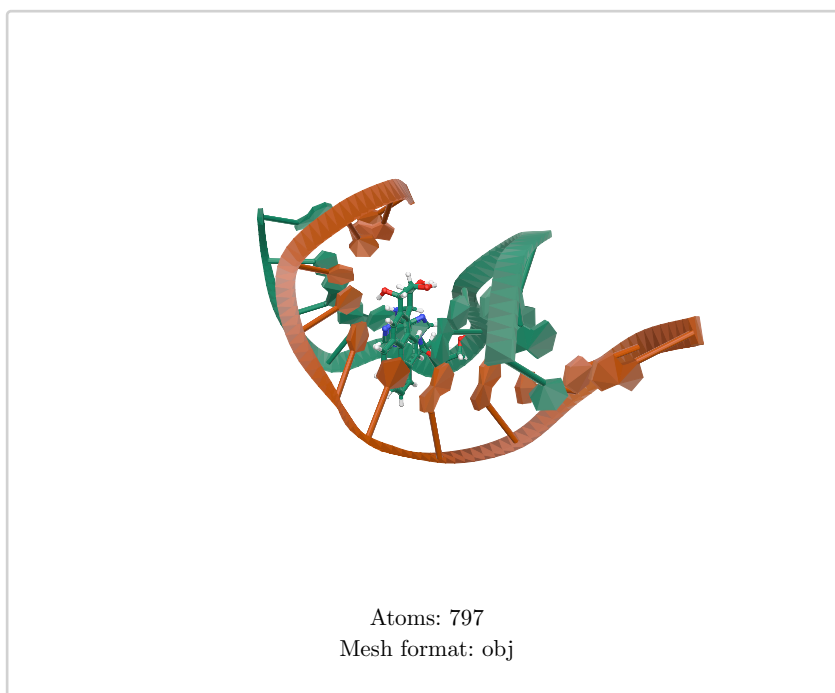


Figure 3: The complete render-object example for 1FYY, including values read from its returned metadata.

Structural data source: RCSB PDB / wwPDB, PDB ID 1FYY, <https://doi.org/10.2210/pdb1FYY/pdb>¹⁴. PDB archive data files are available under CC0 1.0. Primary citation: Volk et al., *Biochemistry* 39, 14040–14053 (2000), doi:10.1021/bi001669l¹⁵.

III.2 Export Commands

#to-obj((data))

Returns OBJ bytes. OBJ is the most useful interchange format when a downstream tool wants text geometry, material references, and group labels.

#to-mtl((data))

Returns the companion MTL material library for OBJ output. Use it alongside **#to-obj** when exporting outside Typst.

#to-stl((data))

Returns binary STL bytes. STL follows Mol* static exporter behavior: the header starts with "Exported from Mol*" and the two-byte facet attribute field is kept at zero.

#to-ply((data))

Returns ASCII PLY bytes. PLY is a Molfig-owned static mesh format because the pinned Mol* geo-export extension does not provide PLY output.

¹⁴<https://doi.org/10.2210/pdb1FYY/pdb>

¹⁵<https://doi.org/10.1021/bi001669l>

Export RCSB PDB entry 1CRN to every mesh format

 `exports.typ`

```

1  #import "@preview/molfig:0.1.4"
2
3  // Structural data: RCSB PDB / wwPDB entry 1CRN.
4  // https://doi.org/10.2210/pdb1CRN/pdb (CC0 1.0)
5  #let data = read("1CRN.bcif", encoding: none)
6  #let obj = molfig.to-obj(data, format: "bcif", representation:
7    "cartoon")
8  #let mtl = molfig.to-mtl(data, format: "bcif", representation:
9    "cartoon")
10 #let stl = molfig.to-stl(data, format: "bcif", representation:
11   "cartoon")
12 #let ply = molfig.to-ply(data, format: "bcif", representation:
13   "cartoon")
14
15 #table(
16   columns: (lfr, lfr),
17   table.header([*Export*], [*Generated bytes*]),
18   [OBJ], [#obj.len()],
19   [MTL], [#mtl.len()],
20   [STL], [#stl.len()],
21   [PLY], [#ply.len()],
22 )

```

Export	Generated bytes
OBJ	602244
MTL	92
STL	1542684
PLY	358830

Figure 4: Byte lengths produced by the complete 1CRN export example.

Structural data source: RCSB PDB / wwPDB, PDB ID 1CRN, <https://doi.org/10.2210/pdb1CRN/pdb>¹⁶. PDB archive data files are available under CC0 1.0. Primary citation: Teeter, *Proceedings of the National Academy of Sciences* 81, 6014–6018 (1984), doi:10.1073/pnas.81.19.6014¹⁷.

¹⁶<https://doi.org/10.2210/pdb1CRN/pdb>

¹⁷<https://doi.org/10.1073/pnas.81.19.6014>

III.3 Metadata Commands

#info({data})

Parses the structure and returns molecular and mesh-planning metadata without maquette rendering. This is the fastest public command for debugging parser, assembly, altLoc, bond, secondary structure, and representation behavior.

#mesh-info({data}, {mesh-format})

Generates a mesh, then delegates mesh metadata extraction to maquette's `get-obj-info`, `get-stl-info`, or `get-ply-info` helper.

Part IV

Shared Mesh Options

The following options are accepted by `#render`, `#render-object`, `#to-obj`, `#to-mtl`, `#to-stl`, `#to-ply`, and `#info`.

IV.1 Input Selection

Option	Default	Meaning
<code>{format}</code>	"auto"	Input parser. Supported values: "auto", "pdb", "cif", "mmCIF", "bcif", "binaryCIF", "binary-cif", "xyz".
<code>{block-index}</code>	none	Zero-based BinaryCIF data block index. Useful when one BinaryCIF file contains multiple data blocks.
<code>{block-header}</code>	" "	Exact BinaryCIF block header. When set, it takes precedence over <code>{block-index}</code> .

Format	Typical extension	Notes
"pdb"	.pdb	Fixed-width text. Keep bytes to preserve columns.
"cif" or "mmCIF"	.cif, .mmCIF	Text CIF categories and loops.
"bcif"	.bcif	BinaryCIF MessagePack data with encoded columns and optional masks.
"xyz"	.xyz	Atom count, comment line, and element/Cartesian-coordinate records. Multiple frames become models; rendering selects the first model, matching Mol* defaults.
"auto"	any	Convenient for exploration, but explicit formats are better for release documents.

BinaryCIF numeric atom-site columns are decoded into typed accessors before falling back to string cells. Molfig does not round-trip BinaryCIF through synthetic CIF text.

IV.2 Structure Selection

Option	Default	Meaning
--------	---------	---------

<code>(assembly)</code>	<code>"1"</code>	Biological assembly id. Use "asymmetric-unit", "none", or an empty string for source coordinates without assembly operators.
<code>(alt-loc)</code>	<code>""</code>	Alternate location id or policy. Supports concrete ids such as "A", plus "highest-occupancy" and "all".
<code>(infer-bonds)</code>	<code>true</code>	Infer covalent bonds when explicit bond data are absent.
<code>(center)</code>	<code>true</code>	Recenters exported vertices around Mol*-style visible export bounds.

Assembly expansion keeps the source model and unit operators as the semantic structure boundary, then materializes mesh coordinates for export. This means metadata can still report selected assembly operators even though OBJ/STL/PLY are static mesh formats.

IV.3 Representation Selection

Option	Default	Meaning
<code>(representation)</code>	<code>"default"</code>	Uses the Mol* Viewer configured default preset, falling back to its automatic size-dependent selection. Use "cartoon" to pin Viewer Quick Styles Cartoon or "polymer-cartoon" for the standalone Cartoon provider.
<code>(color-theme)</code>	<code>"chain-id"</code>	Selects "chain-id", "element-symbol", "entity-id", "operator-name", "plddt-confidence", "qmean-score", or "sb-nbr-partial-charges". Color is serialized through OBJ material references and the companion MTL output. STL and the current PLY output do not carry these colors, so themed document rendering requires mesh-format: "obj".
<code>(theme)</code>	<code>(:)</code>	Applies Mol* Viewer preset overrides. Supported keys are <code>globalName</code> , <code>carbonColor</code> , and <code>symmetryColor</code> . An empty dictionary preserves the preset's normal component-specific themes.

The representation names follow Mol* Viewer Quick Styles rather than only the low-level representation registry. In the Viewer, Cartoon applies the polymer-and-ligand preset; it is not limited to the standalone Cartoon provider. Spacefill is available as "spacefill". Surface corresponds to Mol*'s molecular-surface preset and is selected with representation: "surface". It uses the all component, entity-id colors with the water override, a physical size theme, probeRadius: 1.4, probePositions: 36, and the Mol* CPU marching-cubes path.

"default" and "auto" preserve their distinct public names and follow Mol* size routing for Small, Medium, and Large structures. "default" first applies the Viewer annotation priority described below, while "auto" requests size routing directly. Huge and Gigantic structures use Mol*'s CPU Gaussian-surface branches, including trace-only and structure-level routing where prescribed by ViewerAuto.

IV.3.1 Cartoon, Spacefill, And Surface

The same structure and camera make the geometry difference explicit. Cartoon emphasizes polymer topology and secondary structure; Spacefill exposes atomic packing with van der Waals spheres; Surface shows the continuous solvent-excluded molecular envelope computed by the CPU molecular-surface path.

Compare three representations of RCSB PDB entry 1CRN

 *representations.typ*

```

1  #import "@preview/molfig:0.1.4"
2
3  // Structural data: RCSB PDB / wwPDB entry 1CRN.
4  // https://doi.org/10.2210/pdb1CRN/pdb (CC0 1.0)
5  #let data = read("1CRN.bcif", encoding: none)
6  #let view(rep) = molfig.render(
7    data, format: "bcif", representation: rep,
8    mesh-format: "obj", quality: "high", center: true,
9    output-format: "svg",
10   config: (azimuth: 35, elevation: 24, background: ""),
11   width: 55mm, height: 50mm,
12 )
13 #let panel(label, rep) = [
14   #align(center, strong(label))
15   #block(
16     width: 55mm, height: 50mm, clip: true,
17     align(center + horizon, scale(
18       x: 165%, y: 165%, origin: center + horizon, view(rep),
19     )),
20   )
21 ]
22 #grid(
23   columns: (1fr, 1fr, 1fr), column-gutter: 2mm,
24   panel([Cartoon], "cartoon"),
25   panel([Spacefill], "spacefill"),
26   panel([Surface], "surface"),
27 )

```

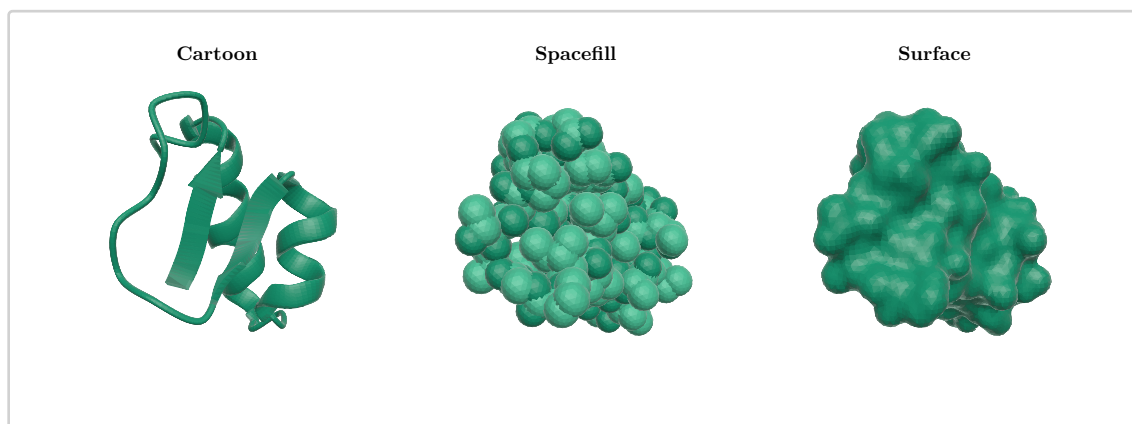


Figure 5: RCSB PDB entry 1CRN rendered with identical camera and quality settings as Cartoon, Spacefill, and Surface.

Structural data source: RCSB PDB / wwPDB, PDB ID 1CRN, <https://doi.org/10.2210/pdb1CRN/pdb>¹⁸. PDB archive data files are available under CC0 1.0. Primary citation: Teeter, *Proceedings of the National Academy of Sciences* 81, 6014–6018 (1984), doi:10.1073/pnas.81.19.6014¹⁹.

IV.3.2 Viewer Theme Overrides

Mol* Viewer presets accept a theme dictionary through their common representation parameters. `globalName` replaces the provider theme for each component, `carbonColor` controls carbon atoms in ball-and-stick ligand, non-standard, branched, and whole-structure element-symbol components, and `symmetryColor` replaces the polymer theme only for non-assembly crystallographic symmetry units. Water, ion, and lipid components keep element-symbol carbon coloring as in the Viewer preset. Individual overrides do not require `globalName` and preserve unaffected component presets, including carbohydrate symbols and illustrative spacefill coloring.

Executable Theme Gallery

Each panel below pairs code with its actual rendering. The code and rendering use the same option dictionaries in `examples/theme-samples.typ`, loaded against this checkout's `lib.typ` and `molfig.wasm`. Recompiling this document checks expected material colors and semantic theme metadata before printing the results; it does not embed cached theme screenshots.

The examples use published data: RCSB PDB 1HHO for chain, entity, operator, element, and carbon coloring; AlphaFold human insulin precursor AF-P01308-F1 (model v6) for pLDDT; the official SWISS-MODEL QMEAN example model_001 for local quality; Mol*'s charge-annotated 7QPD for partial charges; and the existing 1CRN example for symmetry no-op controls. Prediction scores and computed charges are real annotations, not experimental measurements. No coordinates or scores were invented for this gallery. Sources, licenses, and retrieval details are in `examples/data/README.md`. The QMEAN CIF is reproducibly converted from the official processed PDB and JSON, preserving coordinates and full-precision local scores.

¹⁸<https://doi.org/10.2210/pdb1CRN/pdb>

¹⁹<https://doi.org/10.1073/pnas.81.19.6014>

The QMEAN data and rendered panel are adapted from SWISS-MODEL, Computational Structural Biology Group, SIB / University of Basel, under CC BY-SA 4.0²⁰. QMEAN: Benkert et al. (2011), doi:10.1093/bioinformatics/btq662²¹. AlphaFold: Jumper et al. (2021), doi:10.1038/s41586-021-03819-2²², data under CC BY 4.0²³.

Use this shared setup once, then append any panel's render call. The color swatches below each image show its unlit OBJ material colors; lighting may change their appearance on the molecular surface.

Shared data, camera, and export settings

 *theme.typ*

```
1  #import "@preview/molfig:0.1.4"
2
3  // Source files and attribution: examples/data/README.md.
4  #let view = (
5    mesh-format: "obj",
6    quality: "medium",
7    center: true,
8    output-format: "png",
9    config: (
10     azimuth: 35,
11     elevation: 24,
12     projection: "orthographic",
13     width: 768,
14     height: 648,
15     background: "",
16   ),
17   width: 64mm,
18   height: 54mm,
19 )
```

All Seven Global Themes

Every supported color-theme name is shown below. `theme.globalName` makes the global override explicit: unlike the default color-theme: "chain-id", `globalName: "chain-id"` forces chain coloring on every component. Structural themes use ball-and-stick; per-residue annotation themes use cartoon. Explicit representation selection prevents ViewerAuto's annotation priority from replacing the requested theme. Element-symbol additionally sets grey carbon explicitly; its default carbon subtheme is chain-id.

²⁰<https://creativecommons.org/licenses/by-sa/4.0/>

²¹<https://doi.org/10.1093/bioinformatics/btq662>

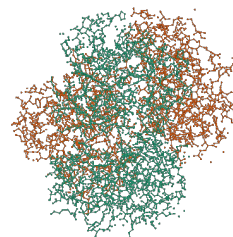
²²<https://doi.org/10.1038/s41586-021-03819-2>

²³<https://creativecommons.org/licenses/by/4.0/>

Chain ID

```
#molfig.render(  
  read("data/1HHO.cif", encoding: none),  
  ..view,  
  format: "cif",  
  representation: "ball-and-stick",  
  assembly: "1",  
  theme: (  
    globalName: "chain-id",  
  ),  
)
```

1HHO oxyhemoglobin: alpha and beta chains have distinct colors, repeated in the second assembly copy.

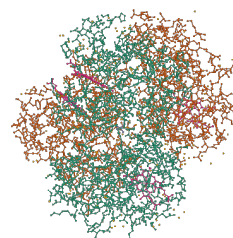


2 material colors

Entity ID

```
#molfig.render(  
  read("data/1HHO.cif", encoding: none),  
  ..view,  
  format: "cif",  
  representation: "ball-and-stick",  
  assembly: "1",  
  theme: (  
    globalName: "entity-id",  
  ),  
)
```

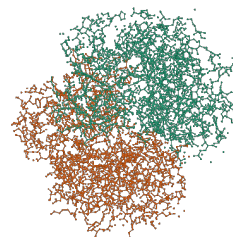
1HHO: alpha/beta proteins, heme, oxygen, phosphate, and water are separate entities. Copies retain entity colors.



6 material colors

Operator name

```
#molfig.render(  
  read("data/IHHO.cif", encoding: none),  
  ..view,  
  format: "cif",  
  representation: "ball-and-stick",  
  assembly: "1",  
  theme: (  
    globalName: "operator-name",  
  ),  
)
```



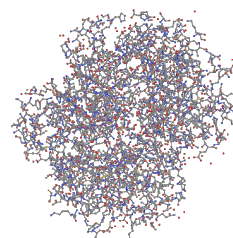
IHHO assembly 1: the identity and twofold-related copies have different colors, regardless of chain or entity.



2 material colors

Element symbol

```
#molfig.render(  
  read("data/IHHO.cif", encoding: none),  
  ..view,  
  format: "cif",  
  representation: "ball-and-stick",  
  assembly: "1",  
  theme: (  
    globalName: "element-symbol",  
    carbonColor: "element-symbol",  
  ),  
)
```



IHHO: carbon is grey, nitrogen blue, and oxygen red. Heme iron, sulfur, and phosphorus keep their element colors.



6 material colors

pLDDT confidence

```
#molfig.render(  
  read("data/AF-P01308-F1-model_v6.cif", encoding:  
    none),  
  ..view,  
  format: "cif",  
  representation: "cartoon",  
  assembly: "asymmetric-unit",  
  theme: (  
    globalName: "plddt-confidence",  
  ),  
)
```

AlphaFold human insulin precursor, model v6: genuine predicted pLDDT spans three confidence bands; no residue exceeds 90.

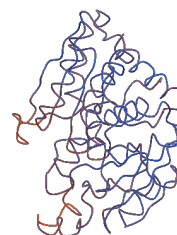


3 material colors

QMEAN score

```
#molfig.render(  
  read("data/qmean-model_001.cif", encoding: none),  
  ..view,  
  format: "cif",  
  representation: "cartoon",  
  assembly: "asymmetric-unit",  
  theme: (  
    globalName: "qmean-score",  
  ),  
)
```

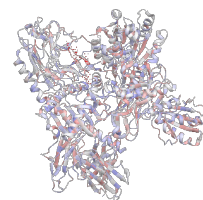
Official SWISS-MODEL QMEAN example model_001. Published local scores are mapped to their original residues, without rounding or rescaling.



142 material colors (12 shown)

SB-NCBR partial charges

```
#molfig.render(  
  read("data/7qpd.fw2.cif", encoding: none),  
  ..view,  
  format: "cif",  
  representation: "cartoon",  
  assembly: "asymmetric-unit",  
  theme: (  
    globalName: "sb-ncbr-partial-charges",  
  ),  
)
```



Mol*'s 7QPD example: computed SQE+qp/Schindler 2021 atomic charges, summed per residue. Negative is red; positive is blue.



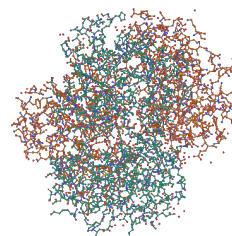
92 material colors (12 shown)

All Three Carbon Subthemes

These examples set carbonColor alone. The chain-id, operator-name, and element-symbol choices change only carbon atoms; nitrogen and oxygen keep their element colors. They exercise the standalone override path without globalName or color-theme.

Carbon: chain ID

```
#molfig.render(  
  read("data/1HH0.cif", encoding: none),  
  ..view,  
  format: "cif",  
  representation: "ball-and-stick",  
  assembly: "1",  
  theme: (  
    carbonColor: "chain-id",  
  ),  
)
```



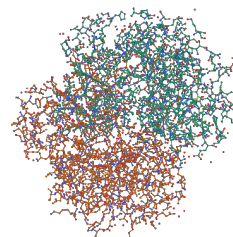
Only carbon follows its chain. Nitrogen and oxygen keep their element colors. No global theme is set.



7 material colors

Carbon: operator name

```
#molfig.render(  
  read("data/1HH0.cif", encoding: none),  
  ..view,  
  format: "cif",  
  representation: "ball-and-stick",  
  assembly: "1",  
  theme: (  
    carbonColor: "operator-name",  
  ),  
)
```



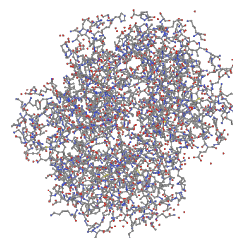
Only carbon distinguishes the two assembly copies. This exercises carbonColor without globalName.



7 material colors

Carbon: element symbol

```
#molfig.render(  
  read("data/1HH0.cif", encoding: none),  
  ..view,  
  format: "cif",  
  representation: "ball-and-stick",  
  assembly: "1",  
  theme: (  
    carbonColor: "element-symbol",  
  ),  
)
```



Carbon becomes grey using only carbonColor; nitrogen and oxygen are unchanged.



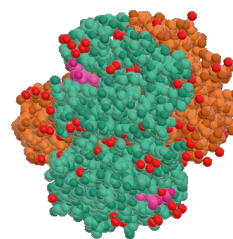
6 material colors

Spacefill Preset Control

A carbon subtheme alone does not replace spacefill's illustrative preset. The first panel is checked against an empty theme dictionary; the second explicitly selects element-symbol coloring with grey carbon.

Spacefill: preset retained

```
#molfig.render(
  read("data/1HH0.cif", encoding: none),
  ..view,
  format: "cif",
  representation: "spacefill",
  assembly: "1",
  theme: (
    carbonColor: "element-symbol",
  ),
)
```



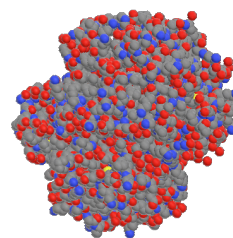
A carbon subtheme alone does not replace the illustrative spacefill preset. Carbon stays lightened by entity.



9 material colors

Spacefill: explicit element theme

```
#molfig.render(
  read("data/1HH0.cif", encoding: none),
  ..view,
  format: "cif",
  representation: "spacefill",
  assembly: "1",
  theme: (
    globalName: "element-symbol",
    carbonColor: "element-symbol",
  ),
)
```



Select element-symbol explicitly to obtain grey carbon in spacefill. Geometry and camera are unchanged.



6 material colors

Assembly Is Not Crystal Symmetry

The three panels below must match. They contain polymer geometry and assembly operators, so they detect accidental application of `symmetryColor` to a biological assembly, both alone and with `globalName`. The build compares the complete OBJ geometry/material assignments with the baseline. These controls use the bundled RCSB PDB 1CRN biological assembly 1.

The public Typst API currently has no crystal-contacts or crystallographic symmetry-expansion option. These panels are negative controls for biological assemblies, not positive examples of a generated crystal-symmetry scene. In contrast, the global operator-name sample above intentionally colors assembly copies by operator.

Assembly: baseline

```
#molfig.render(  
  read("data/1crn.bcif", encoding: none),  
  ..view,  
  format: "bcif",  
  representation: "cartoon",  
  assembly: "1",  
  theme: (:),  
)
```

RCSB PDB 1CRN, assembly 1: cartoon geometry with its preset chain color.




1 material color

Assembly: symmetryColor only

```
#molfig.render(  
  read("data/1crn.bcif", encoding: none),  
  ..view,  
  format: "bcif",  
  representation: "cartoon",  
  assembly: "1",  
  theme: (  
    symmetryColor: "operator-name",  
  ),  
)
```

The rendering must equal the baseline. Biological assembly operators are not crystal-symmetry operators.




1 material color

Assembly: global + symmetry

```
#molfig.render(
  read("data/1crn.bcif", encoding: none),
  ..view,
  format: "bcif",
  representation: "cartoon",
  assembly: "1",
  theme: (
    globalName: "chain-id",
    symmetryColor: "qmean-score",
  ),
)
```



The rendering must still equal the baseline. An inapplicable symmetry override must not replace the chain theme with missing-score grey.



1 material color

To rerun the standalone gallery and its checks from the repository root:

```
typst compile --root . package/examples/theme.typ package/examples/
theme.pdf
```

IV.3.3 Annotation Color Rules

With representation: "default", Molfig follows the ViewerAuto preset and selects the first applicable embedded annotation in this order: plddt-confidence, qmean-score, then sb-ncbr-partial-charges. These names may also be selected explicitly with (color-theme). representation: "auto" skips annotation selection and performs only structure-size routing.

Theme	Value rule	Colors
plddt-confidence	score <= 50, <= 70, <= 90, then > 90	#ff7d45 , #ffdb13 , #65cbf3 , #0053d6
qmean-score	Values through 0.5 use orange; 0.5..1.0 interpolates to blue.	#ff5000 to #025afd
sb-ncbr-partial-charges	Residue charge: negative through zero to positive.	#ff0000 through #ffffff to #0000ff

Missing pLDDT values fall back to B_iso_or_equiv. Unavailable or negative quality scores use #aaaaaa . Missing partial-charge locations use #66ff00 , matching the Mol* provider.

Annotation colors require OBJ when rendered through maquette because STL and the current PLY schema have no material color channel.

IV.3.4 Chain ID Color Rules

The "chain-id" theme follows these rules:

The Viewer "spacefill" preset intentionally overrides this option with Mol* illustrative entity-id coloring. It lightens carbon colors in CIE Lab and forces water entities to ■ #ff0d0d.

- For atomic models, the author chain id (auth_asym_id) is used when it is present; otherwise the label chain id (label_asym_id) is used. PDB input normally has the same value for both. Coarse IHM spheres and Gaussians use their asym id.
- Unique chain ids are collected in model order. Atomic chains are collected first, followed by previously unseen coarse-sphere and coarse-Gaussian chain ids. Assembly copies retain the source chain id and therefore retain its color.
- Colors are assigned from the following 25-color Mol* many-distinct palette. If a structure contains more than 25 chain ids, the palette repeats from the first color.

Palette positions	Colors
1–5	■ #1b9e77, ■ #d95f02, ■ #7570b3, ■ #e7298a, ■ #66a61e
6–10	■ #e6ab02, ■ #a6761d, ■ #666666, ■ #e41a1c, ■ #377eb8
11–15	■ #4daf4a, ■ #984ea3, ■ #ff7f00, ■ #ffff33, ■ #a65628
16–20	■ #f781bf, ■ #999999, ■ #66c2a5, ■ #fc8d62, ■ #8da0cb
21–25	■ #e78ac3, ■ #a6d854, ■ #ffd92f, ■ #e5c494, ■ #b3b3b3

Chain-associated geometry, including cartoon, ribbon, backbone, nucleotide, carbohydrate, gap, and coarse visuals, uses the assigned chain color unless the visual has an explicit atomic material. Atomic visuals apply a chain-and-element rule:

- Carbon uses its chain color for ordinary atoms, polymers, ligands, and branched components.
- Water, ion, and lipid components use element-symbol colors for every atom, including carbon.
- Non-carbon atoms use their element-symbol color. The symbol is read from type_symbol when available, otherwise from the parsed element field. Unknown symbols fall back to white.
- Bond geometry inherits the material of its source atom. Component visuals that emit two directed half-bonds consequently color each half from its adjacent atom.

Element-symbol colors include Mol*'s default lightness adjustment. The final RGB values written to OBJ materials are:

Elements	Final colors
Hydrogen isotopes	H ■ #ffffff, D ■ #ffffca, T ■ #ffffaa
Common organic elements	C ■ #999999, N ■ #4259ff, O ■ #ff2618, P ■ #ff8a14, S ■ #ffff3e, Se ■ #ffab17

Halogens	F  #9aea5a , Cl  #37fb2e , Br  #b13431 , I  #9e179e
Alkali and alkaline-earth metals	Na  #b566fd , Mg  #95ff1f , K  #994ade , Ca  #4eff1e
Transition metals	Mn  #a683d1 , Fe  #eb703c , Co  #fb9aaa , Ni  #5cda5a , Cu  #d3893c , Zn  #8689ba
Other or unknown	 #ffffff

The companion MTL records opacity 0.3 for branched components, 0.6 for water and lipids, and 1.0 otherwise. Molfig's automatic maquette material map currently forwards RGB colors only; it does not forward these MTL opacity values.

Value	Best for	Main geometry	Notes
"default"	Viewer-compatible automatic figures	ViewerAuto annotation theme, then size-selected geometry	Uses pLDDT, QMEAN, or SB-NCBR partial charges when applicable, with Gaussian surfaces for Huge/Gigantic structures.
"auto"	Explicit automatic selection	Atomic detail, polymer Cartoon, or a Gaussian surface according to structure size	Huge/Gigantic routing follows the ViewerAuto size thresholds.
"cartoon"	General figures	Polymer Cartoon plus atomic detail for non-polymers and carbohydrate visuals	Pins the Mol* Viewer Quick Styles Cartoon preset.
"polymer-cartoon"	Polymer-only figures	Polymer trace, nucleotide ring, and polymer gap visuals	Standalone Mol* Cartoon provider defaults.
"spacefill"	Atomic packing	Atom spheres	Viewer Quick Styles Spacefill with illustrative entity-id colors and water override.
"surface"	Solvent-accessible shape	CPU molecular-surface field and marching-cubes mesh	Viewer Quick Styles Surface with entity-id colors and red water override.
"ball-and-stick"	Ligands and small molecules	Atom spheres plus bond cylinders	Good for local chemistry and explicit bond inspection.
"ribbon"	Backbone shape	Ribbon-oriented polymer geometry	Good for broad fold visibility.

"backbone"	Trace-only views	Backbone cylinders and spheres	Lower-detail alternative for polymer paths.
------------	------------------	--------------------------------	---

IV.4 Geometry And Quality

Option	Default	Meaning
{quality}	"custom"	"custom" preserves explicit numeric controls. "auto" chooses a preset from structure size. Also accepts "highest", "higher", "high", "medium", "low", "lower", "lowest".
{decimate}	0	Molfig-side molecular level-of-detail strength in [0, 1]. It reduces sphere detail, polymer curve/profile segments, surface resolution, probe sampling, and exported cylinder detail before maquette receives the mesh. In <code>#render</code> and <code>#render-object</code> , <code>config.decimate</code> is also consumed this way and is not forwarded to maquette's generic triangle decimator.
{sphere-detail}	2	Sphere tessellation detail. Public input is clamped by the plugin.
{linear-segments}	8	Curve subdivisions for polymer and tube paths.
{radial-segments}	16	Radial subdivisions for cylinders, tubes, and ribbon-like profiles.
{radius-scale}	1.0	Global radius multiplier.
{atom-radius}	0.28	Explicit atom radius for simple representations.
{bond-radius}	0.12	Explicit bond radius for cylinder-based bond visuals.
{ribbon-radius}	0.2	Tube/ribbon radius control.
{ribbon-width}	0.55	Ribbon width control.
{helix-profile}	"elliptical"	"elliptical", "rounded", or "square". "sheet" is accepted as an alias for square.
{round-cap}	false	Enable rounded caps where supported by the selected cartoon/ribbon path.
{sheet-arrow-factor}	1.5	Scales sheet arrow geometry.
{tubular-helices}	false	Prefer tubular helix geometry in supported cartoon paths.

Quality presets override the explicit numeric tessellation controls. Use quality: "custom" when exact reproducibility matters, and use quality: "auto" or a lower preset when large structures compile too slowly. `<decimate>` is applied after the quality preset, so it can be used as an additional opt-in geometry budget without changing representation selection.

Preset	Sphere de-tail	Radial segments	Linear segments
"highest"	3	36	18
"higher"	3	28	14
"high"	2	20	10
"medium"	1	12	8
"low"	0	8	3
"lower"	0	4	2
"lowest"	0	2	1

IV.4.1 Semantic Decimate

`<decimate>` is intended for document builds where a molecularly meaningful lower-detail mesh is preferable to generic triangle clustering. The example below keeps the same PDB entry, quality preset, and camera, but compares practical semantic level-of-detail strengths for Cartoon, Surface, and Spacefill. Cartoon primarily reduces curve and profile segments, Surface raises the molecular-surface grid resolution and reduces probe sampling, and Spacefill reduces sphere tessellation before maquette receives the OBJ.

Compare Molfig semantic decimate strengths on RCSB PDB entry 1CRN  *decimate.typ*

```

1  #import "@preview/molfig:0.1.4"
2
3  // Structural data: RCSB PDB / wwPDB entry 1CRN.
4  // https://doi.org/10.2210/pdb1CRN/pdb (CC0 1.0)
5  #let data = read("1CRN.bcif", encoding: none)
6  #let panel-width = 55mm
7  #let panel-height = 42mm
8
9  #let panel(name, representation, strength, zoom) = [
10   #align(center)[#strong(name)\
11     #text(size: 8pt)[decimate: #strength]]
12   #block(width: panel-width, height: panel-height, clip: true)[
13     #align(center + horizon, scale(
14       x: zoom, y: zoom, origin: center + horizon,
15       molfig.render(data, format: "bcif", representation:
16         representation,
17         mesh-format: "obj", quality: "high", center: true,
18         output-format: "svg",
19         config: (azimuth: 35, elevation: 24, background: "",
20           decimate: strength),
21         width: panel-width, height: panel-height),
22     ))
23   ]
24
25  #grid(
26    columns: (1fr, 1fr, 1fr), column-gutter: 2mm, row-gutter: 3mm,
27    panel([Cartoon], "cartoon", 0, 165%),
28    panel([Cartoon], "cartoon", 0.35, 165%),
29    panel([Cartoon], "cartoon", 0.65, 165%),
30    panel([Surface], "surface", 0, 145%),
31    panel([Surface], "surface", 0.35, 145%),
32    panel([Surface], "surface", 0.65, 145%),
33    panel([Spacefill], "spacefill", 0, 145%),
34    panel([Spacefill], "spacefill", 0.35, 145%),
35    panel([Spacefill], "spacefill", 0.65, 145%),
36  )

```

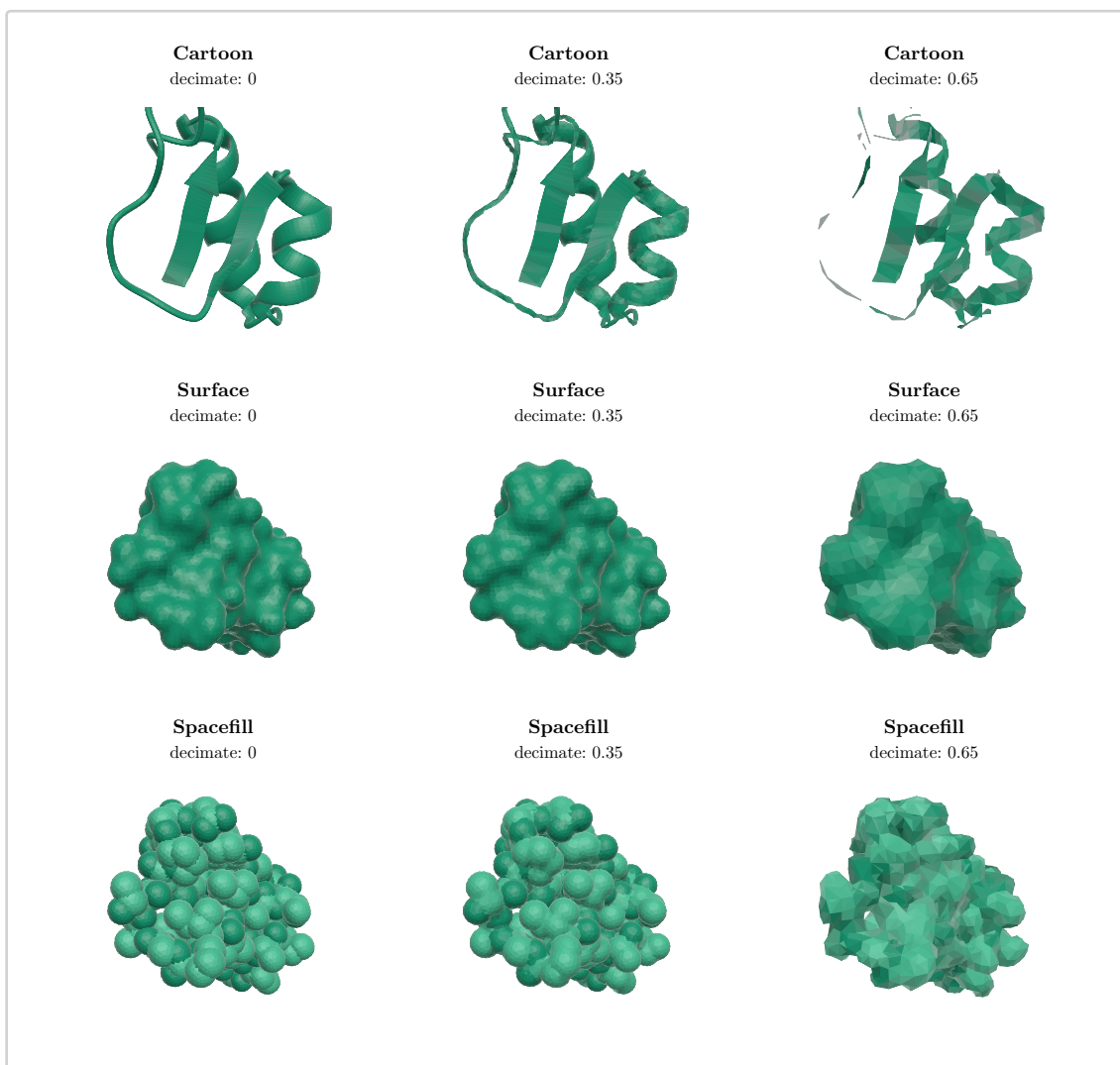


Figure 6: RSCB PDB entry 1CRN rendered as Cartoon, Surface, and Spacefill with practical Molfig semantic decimate strengths.

Structural data source: RSCB PDB / wwPDB, PDB ID 1CRN, <https://doi.org/10.2210/pdb1CRN/pdb>²⁴. PDB archive data files are available under CC0 1.0. Primary citation: Teeter, *Proceedings of the National Academy of Sciences* 81, 6014–6018 (1984), doi:10.1073/pnas.81.19.6014²⁵.

The public default is quality: "custom", so the numeric defaults in this manual are honored unless you explicitly choose a preset or "auto".

²⁴<https://doi.org/10.2210/pdb1CRN/pdb>

²⁵<https://doi.org/10.1073/pnas.81.19.6014>

Part V

Maquette Rendering

`#render` calls one of maquette's mesh renderers based on `<mesh-format>`: `render-obj`, `render-stl`, or `render-ply`. The `<config>` dictionary is JSON-encoded and passed through. For OBJ, Molfig derives maquette's materials dictionary from the exported material ids so `color-theme: "chain-id"` is visible in the document. User-supplied material entries are merged last and therefore take precedence. STL has no material channel, and Molfig's current PLY schema contains no vertex or face color properties. Consequently, maquette renders STL and PLY without the selected color theme.

The complete 1FYY `render-object` example above demonstrates camera, background, dimensions, and raster-output passthrough, and its rendered result shows the exact content returned in `object.content`.

Molfig does not validate maquette-specific keys. Unknown keys are passed to maquette, so maquette remains the source of truth for camera and renderer settings.

V.1 Choosing A Mesh Format

Format	Command	Strength	Tradeoff
OBJ	<code>#to-obj</code>	Readable text, group labels, material references	Larger than binary STL and needs MTL when material rows are consumed externally.
MTL	<code>#to-mtl</code>	Companion material rows for OBJ	Not a geometry format by itself.
STL	<code>#to-stl</code>	Compact binary triangle stream	No stable text metadata channel; facet attribute bytes are zero.
PLY	<code>#to-ply</code>	Text mesh with Molfig group metadata	Package-owned output, not a pinned Mol* exporter format.

For visual documents, start with OBJ. Switch to STL only when the downstream consumer specifically wants binary STL, and use PLY when the group metadata is more useful than OBJ/MTL compatibility.

Part VI

Metadata Reference

`#info` returns a dictionary. It is intended for document logic, regression tests, and debugging. The exact set of keys may grow as Molfig's Mol*-parity layer grows, but the current major groups are:

Key	Type	Meaning
atom_count	integer	Visible atom count after selected assembly/alt-Loc handling and geometry expansion.
bond_count	integer	Visible bond count used by static geometry.
alt_locs	array	Available alternate location ids from the source coordinates.
alt_locs_info	dictionary	Selected altLoc policy and available ids.
assemblies	array	Available biological assembly summaries.
assembly	dictionary	Selected assembly id.
source_data	dictionary	Input source categories, row counts, and original source kind.
structure	dictionary	Model/Structure/Unit counts, boundary, conformation, segments, ranges, and lookup3d summary.
secondary_structure	dictionary	Helix and sheet ranges.
representation	dictionary	Selected and realized visual names.
render_objects	array	Semantic render-object spans with geometry type, visual, chain, residue range, group id, component, representation tag/order, provider color-theme metadata, and value-cell style counts.
bond_metadata	dictionary	Counts by bond source and flags such as struct_conn, index_pair, chem_comp, aromatic, and resonance.
bounds	dictionary	Expanded coordinate bounds before export centering.

Inspect the Model/Structure/Unit result for RCSB PDB entry 9R1O

 *metadata.typ*

```

1  #import "@preview/molfig:0.1.4"
2
3  // Structural data: RCSB PDB / wwPDB entry 9R1O.
4  // https://doi.org/10.2210/pdb9R1O/pdb (CC0 1.0)
5  #let meta = molfig.info(
6    read("9R1O.pdb", encoding: none),
7    format: "pdb",
8    representation: "cartoon",
9    assembly: "1",
10   alt-loc: "highest-occupancy",
11 )
12
13 #table(
14   columns: (lfr, lfr),
15   table.header([*Property*], [*Value*]),
16   [Atoms], [#meta.atom_count],
17   [Bonds], [#meta.bond_count],
18   [Assembly], [#meta.assembly.id],
19   [Units], [#meta.structure.unit_count],
20   [Realized visuals], [#meta.representation.realized_visuals.join(",
21   ")]],
21 )

```

Property	Value
Atoms	1503
Bonds	12
Assembly	1
Units	2
Realized visuals	polymer-trace, polymer-gap, element-sphere, intra-bond

Figure 7: Selected structure and representation metadata produced from PDB entry 9R1O.

Structural data source: RCSB PDB / wwPDB, PDB ID 9R1O, <https://doi.org/10.2210/pdb9R1O/pdb>²⁶. PDB archive data files are available under CC0 1.0. Deposition authors: Petrenas, Ozga, Chubb, and Woolfson.

²⁶<https://doi.org/10.2210/pdb9R1O/pdb>

The `render_objects` array can be used for stronger assertions, for example checking that at least one object has `secondary_type == "helix"` or that `realized_visuals` contains `polymer-trace`.

VI.1 Assembly And Unit Metadata

Assembly selection is visible through both `info(...).structure` and mesh export metadata. OBJ and PLY include `molfig_operator_metadata` comments when an assembly is selected. STL has no equivalent text channel, so assembly operator metadata should be inspected through `#info` or `#render-object`.

Path	Example	Meaning
<code>structure.unit_count</code>	<code>meta.structure.unit_count</code>	Number of units after structure construction.
<code>structure.symmetry_group_count</code>	<code>meta.structure.symmetry_group_count</code>	Mol*-style symmetry grouping count.
<code>structure.coordinate_system</code>	<code>meta.structure.coordinate_system</code>	Selected coordinate operator summary.
<code>structure.boundary</code>	<code>meta.structure.boundary.sphere_radius</code>	Boundary sphere and box used by structure-level geometry.
<code>structure.lookup3d</code>	<code>meta.structure.lookup3d.unit_count</code>	Lookup3D summary for constructed units.

Part VII

Practical Workflows

VII.1 A Publication Figure

The complete 9R1O Quickstart is the publication-oriented example: it pins the PDB entry, biological assembly, representation, tessellation quality, camera, centering, mesh format, and raster output. Its code, rendered PDF, and source attribution are shown together in the Quickstart section.

VII.2 A Lightweight Draft Figure

Draft a large real assembly from RCSB PDB entry 9M1U

 9M1U.typ

```
1  #import "@preview/molfig:0.1.4"
2
3  // Structural data: RCSB PDB / wwPDB entry 9M1U.
4  // https://doi.org/10.2210/pdb9M1U/pdb (CC0 1.0)
5  #molfig.render(
6    read("9M1U.pdb", encoding: none),
7    format: "pdb",
8    representation: "cartoon",
9    assembly: "1",
10   quality: "auto",
11   mesh-format: "obj",
12   output-format: "png",
13   config: (elevation: 45, background: ""),
14 )
```

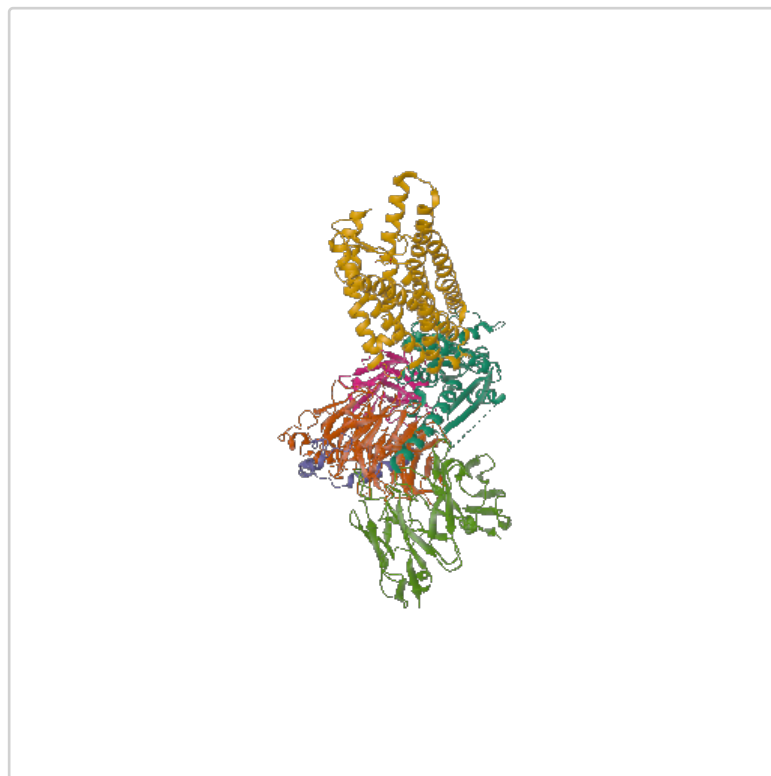


Figure 8: RCSB PDB entry 9M1U rendered as a Cartoon with automatic quality and PNG output.

Structural data source: RCSB PDB / wwPDB, PDB ID 9M1U, <https://doi.org/10.2210/pdb9M1U/pdb>²⁷. PDB archive data files are available under CC0 1.0. Structure authors: Liu, Zhang, and Xu. Primary citation: Zhang et al. (2026), *The EMBO Journal*, doi:10.1038/s44318-026-00823-y²⁸.

VII.3 Export For External Tools

The complete 1CRN export example in the Export Commands section generates OBJ, MTL, STL, and PLY bytes from a real BinaryCIF entry and typesets their measured sizes. Typst does not write arbitrary files from a package call; use those byte values inside document logic or expose them through a workflow that may write artifacts outside Typst.

²⁷<https://doi.org/10.2210/pdb9M1U/pdb>

²⁸<https://doi.org/10.1038/s44318-026-00823-y>

Part VIII

Troubleshooting

Symptom	Likely cause	Action
Plugin says Molfig expects bytes.	The input was not bytes, inline string data, or a Typst 0.15+ path value.	For example, pass <code>path("9R10.pdb")</code> on Typst 0.15+, or <code>read("9R10.pdb", encoding: none)</code> for Typst 0.14-compatible documents.
BinaryCIF reports a missing encoding.	The file is not valid BinaryCIF MessagePack or a column lacks required BinaryCIF encoding metadata.	Check the source file and use format: "cif" only for text CIF/mmCIF.
The figure is sparse or missing chains.	Assembly or alt-Loc selection filtered the structure.	Inspect <code>molfig.info(...).assemblies</code> and <code>alt_locs_info</code> , then set <code>{assembly}</code> and <code>{alt-loc}</code> explicitly.
A large assembly compiles slowly.	High tessellation or too much assembly geometry.	Use <code>quality: "auto", {decimate}</code> , lower <code>{radial-segments}</code> and <code>{linear-segments}</code> , or choose a lighter representation.
SVG parsing fails with "nodes limit reached".	A high-poly mesh, commonly a large spacefill representation, expands to more SVG nodes than Typst accepts.	Use <code>output-format: "png"</code> . If vector output is required, reduce <code>{quality}</code> , <code>{sphere-detail}</code> , <code>{linear-segments}</code> , or <code>{radial-segments}</code> .
Bonds are missing.	The file lacks explicit bonds and inference is disabled.	Leave <code>{infer-bonds}</code> as <code>true</code> , or inspect <code>bond_metadata</code> to see available bond sources.
Rendered view differs from external OBJ inspection.	Different camera, centering, or mesh format.	Pin <code>{center}</code> , <code>{mesh-format}</code> , <code>maquette {config}</code> , and representation quality options.

For reproducible documents, follow the 9R1O Quickstart pattern: use a stable archive identifier, explicit input and mesh formats, a pinned representation, assembly, quality, centering policy, output format, and camera.

Part IX

License And Notices

Molfig package code is distributed under the MIT License. The package also contains third-party material that is covered separately:

Material	Scope	License / notice
Mol*	molfig.wasm contains Rust behavior and generated reference data derived from Mol* parity work.	MIT License; copyright (c) 2017 - now, Mol* contributors.
PDB archive data	examples/data/*.pdb, examples/data/*.cif, examples/data/*.bcif, and generated example PDFs.	CC0 1.0 Universal Public Domain Dedication; attribution to PDB IDs and structure authors is encouraged.
PubChem3D examples	examples/data/*.xyz, the validation manifest, and generated PDF/PNG.	PubChem-generated conformer coordinates for CIDs 702, 241, 2244, and 2519. PubChem makes generated information available without cost and without restriction; see the recorded NCBI/PubChem policies and attribution.
Typst helper packages	maquette for rendering and mantys for this manual.	Imported by Typst; not vendored by Molfig.

The package includes NOTICE.md and THIRD_PARTY_NOTICES.md with the full distribution notice. Example data attributions are also listed in examples/data/README.md and examples/data/ATTRIBUTION.tsv.

For figures made from PDB archive data, cite the PDB ID and DOI. For example:

Suggested data attribution

- 1 Structural data source: RCSB PDB / wwPDB, PDB ID 9R10,
- 2 <https://doi.org/10.2210/pdb9R10/pdb>.
- 3 PDB archive data files are available under CC0 1.0.

For a bundled XYZ record, cite its PubChem record and the PubChem service. For example:

Suggested XYZ example attribution

- 1 Coordinate source: PubChem CID 702 (ethanol), PubChem3D conformer
- 2 000002BE00000001, retrieved through PubChem PUG REST on 2026-08-24.
- 3 <https://pubchem.ncbi.nlm.nih.gov/compound/702>
- 4 <https://doi.org/10.1093/nar/gkae1059>

When describing Mol* parity or comparing against Mol* output, cite Mol*:

Suggested Mol* citation

- 1 Sehna1 et al. Mol* Viewer: modern web app for 3D visualization and analysis of large biomolecular structures.
- 2 Nucleic Acids Research 49:W431-W437 (2021).
- 3 <https://doi.org/10.1093/nar/gkab314>

No endorsement by Mol*, RCSB PDB, wwPDB, structure authors, or data providers is implied.

Part X

Development

The package consists of a Typst wrapper, a Rust WebAssembly plugin, tests, and documentation:

Path	Role
package/lib.typ	Public Typst API and maquette bridge.
wasm-plugin/src/	Rust parser, Model/Structure/Unit layer, geometry builders, exporters, and Typst plugin ABI.
wasm-plugin/tests/	Rust regression tests and Typst contract/integration tests.
package/docs/documentation.typ	This Mantys manual.
package/docs/documentation.pdf	Compiled reference manual.
package/molfig.wasm	Checked-in WebAssembly plugin consumed by Typst.

```
cd wasm-plugin
cargo fmt --check
cargo test
node tests/validate-pubchem-xyz.mjs
cargo build --release --target wasm32-unknown-unknown
cp target/wasm32-unknown-unknown/release/molfig.wasm ../package/molfig.wasm
cd ../package
just docs
```

Regenerate package/molfig.wasm after Rust changes that affect the Typst plugin. Regenerate this PDF after documentation or public API changes.

Part XI

Index

I

`#info` 16

M

`#mesh-info` 16

R

`#render` 11

`#render-object` 12

T

`#to-mtl` 14

`#to-obj` 14

`#to-ply` 14

`#to-stl` 14